10

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Double-Checked Locking (DCL) & Singleton Tuzağı: volatile Neden Zorunlu?

Double-Checked Locking (DCL) & The Singleton Trap: Why volatile is Non-Negotiable

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              DOUBLE-CHECKED LOCKING (DCL) NESNE OLUŞTURMA ADIMLARI              │
└──────────────────────────────────────────────────────────┘

`instance = new Singleton();` İŞLEMİ TEK BİR ADIM DEĞİLDİR!

GERÇEK KOD ADIMLARI:
1. memory = allocate();   // Bellekte yer tahsis et
2. ctorSingleton(memory); // Constructor'ı çalıştır (Alanları doldur)
3. instance = memory;     // Referansı instance değişkenine bağla

CPU / JIT YENİDEN SIRALAMASI (REORDERING):
1. memory = allocate();   // Bellek tahsis edildi
3. instance = memory;     // ⚠ DİKKAT! Referans erkenden bağlandı! (İçi henüz BOŞ!)
2. ctorSingleton(memory); // Constructor hala çalışıyor...

THREAD 2 TAM BU ANDA GELİR:
if (instance == null) ──→ FALSE DÖNER! (Çünkü 3. adım çalıştı!)
return instance;      ──→ 💣 GÜM! Thread 2 henüz İÇİ BOŞ nesneyi kullanmaya başlar!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir mobilyacıdan gardırop siparişi verdiniz. Usta dükkanda parçaları birleştirip öyle teslim etmelidir. Ama usta hızlanmak için boş gardırop kolisini (Bellek tahsisi) evinize getirip kapıya koyar (Referansı bağlar), raflarını ve kapaklarını ise sonradan takmaya başlar. Tam o sırada evin çocuğu gelip kapağı olmayan, vidaları takılmamış gardırobun içine kıyafet koymaya çalışırsa gardırop çocuğun üstüne devrilir! İşte 'volatile' koymazsanız, Java nesnenin bitmemiş kolisini diğer thread'lere dolu gibi gösterir!

🇬🇧 ENGLISH (ELI5)

You order an assembled oak wardrobe. The carpenter should assemble the doors and drawers before handing over the keys. Instead, the rushed carpenter sets a hollow wooden frame in your bedroom (instance pointer published) while rummaging for screws and shelves in the hallway (constructor executing). If your child immediately tosses coats inside, the unfinished frame collapses! Without `volatile`, the JVM publishes the object reference BEFORE constructor completion, causing threads to access half-baked objects!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

DCL Probleminin Çözümü ve `volatile`: 1) Neden synchronized yetersiz?: Metodun tamamını `synchronized` yapmak her okumada kilit maliyeti yaratır. Bu yüzden 'önce kilitsiz kontrol et, null ise kilide gir, içeride bir daha kontrol et' (Double-Check) tasarlandı. 2) Yarı Yapılandırılmış Nesne Kaçağı (Partially Constructed Object): JIT derleyicisi 2. ve 3. adımların yerini değiştirebilir (Instruction reordering). Thread A henüz constructor'ı bitirmeden `instance` referansı non-null hale gelir. O anda kilit dışındaki Thread B ilk kontrolde `instance != null` görür ve nesneyi alır; ancak nesnenin alanları henüz varsayılan `0` veya `null` değerindedir! 3) volatile Çözümü: `private static volatile Singleton instance;` tanımlandığında, StoreStore ve StoreLoad bariyerleri sayesinde nesnenin constructor'ı tamamen bitmeden referans ASLA yayınlanamaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Deconstructing Double-Checked Locking: 1) The Performance Impetus: Synchronizing the entire `getInstance()` method incurs mutex acquisition on every single read. DCL seeks to synchronize strictly on the initial null check. 2) Reordering Vulnerability: The JVM allows reordering between step 2 (instance initialization) and step 3 (pointer assignment). Thread A executes step 1 then step 3; Thread B executes the outer unsynchronized check, observes `instance != null`, and happily returns the instance while Thread A is still executing step 2! Thread B observes a partially constructed object, crashing with NPEs or corrupt data. 3) The Volatile Fix: Declaring `volatile Singleton instance` forces a StoreStore barrier prior to reference assignment, ensuring all constructor writes flush before publication.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Güvenli Double-Checked Locking (DCL) Singleton
public class SafeDclSingleton {
    // ⚠ KRİTİK: volatile OLMADAN DCL %100 HATALIDIR!
    private static volatile SafeDclSingleton instance;
    private final String connectionString;

    private SafeDclSingleton() {
        // Ağır başlatma simülasyonu
        this.connectionString = "jdbc:postgresql://db.maang.internal:5432/core";
    }

    public static SafeDclSingleton getInstance() {
        // 1. Kontrol (Kilitsiz - Hızlı Okuma)
        SafeDclSingleton localRef = instance;
        if (localRef == null) {
            synchronized (SafeDclSingleton.class) {
                // 2. Kontrol (Kilit İçinde - Kesin Tekillik)
                localRef = instance;
                if (localRef == null) {
                    instance = localRef = new SafeDclSingleton();
                }
            }
        }
        return localRef;
    }

    public String getConnectionString() {
        return connectionString;
    }

    void main() {
        var s = SafeDclSingleton.getInstance();
        IO.println("Bağlantı Başarılı: " + s.getConnectionString());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı Tuzağı: Mülakatçı size DCL kodunu tahtaya yazar ve 'Burada volatile'ı unutsak ne olur?' der. 'NullPointerException veya beklenmedik değerler alırız çünkü başka bir thread henüz constructor'ı tamamlanmamış yarı-yapılandırılmış (partially constructed) bir nesneye erişir' cevabını vermeyen aday elenir!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

The 1 MAANG Concurrency Question: 'What breaks if we omit volatile in Double-Checked Locking?' Answer: 'Threads will observe a partially constructed object! The compiler or CPU can reorder memory assignment before constructor invocation, exposing uninitialized fields to unsynchronized reader threads.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- DCL deseninde volatile kullanımı bir seçenek değil, matematiksel bir zorunluluktur.
- volatile olmadan başka bir thread yarı-yapılandırılmış (partially constructed) nesne görür.
- Modern Java'da alternatif olarak 'Initialization-on-demand holder' idiom tercih edilebilir.

🎯 Key Takeaways for the Interview

- volatile in Double-Checked Locking is a strict requirement, not an optional optimization.
- Without volatile, reader threads risk observing uninitialized, partially constructed state.
- Alternatively, use the Bill Pugh Initialization-on-demand holder idiom.